RAFFLES INSTITUTION
H2 Computing (9569)

2026 Year 6

Chapter S3E: NoSQL Database

CONTENT

1. Introduction to NoSQL

Relational databases (commonly referred to as SQL databases) work well with structured data since each table's schema (the precise description of the data to be stored and the relationships between them) is always clearly defined. However, with the increasing number of ways to gather and generate data, we often need to deal with unstructured or semi-structured data. For example, a convenience store that frequently refreshes the services it provides may sell both mobile phones as well as groceries. To run the store, information about both mobile phones (e.g., model names and prices) and groceries (e.g., prices and descriptions) need to be stored. In the future, the store may also start selling mobile phone subscription plans as well. Storing all these data in the same relational database may not be easy. In this case, non-relational databases, also referred to as NoSQL databases, can offer a better choice.

There are four main types of NoSQL databases: key-value databases, document databases, wide-column databases and graph databases. For our syllabus, we focus on MongoDB, a type of *document database*. Conceptually, document databases provide key-based access to data, where each key maps to a document containing multiple fields. These documents are typically stored in a JSON-like format, making them easy to represent and modify.

2. Addressing shortcomings of relational databases

Relational databases have a predefined schema that is difficult to change. Even if you wish to add a field to a small number of records, you still need to include the field for the entire table. Therefore, it can be difficult to support the processing of unstructured data using relational databases compared to NoSQL databases.

Relational databases do not naturally support hierarchical data storage, which involves nested or tree-like structures. Representing such hierarchical data in a relational database often requires multiple tables and joins, making the database design and queries more complex. In contrast, NoSQL databases such as document databases can store hierarchical data directly within documents, making them more suitable for applications that involve nested data.

Relational databases are mainly vertically scalable while NoSQL databases are mainly horizontally scalable. Vertically scalable means that improving the performance of a relational database server usually requires upgrading an existing server with faster processors and more memory. Such high-performance components can be expensive, and upgrades are limited by the capacity of a single machine. On the other hand, horizontally scalable means that the performance of a NoSQL database can be improved by simply increasing the number of servers. This is relatively cheaper as mass-produced average-performance computers are easily available at low prices.

Relational databases are typically hosted on a single server, which can make the database unavailable if that server fails. NoSQL databases are designed to take advantage of multiple servers so that if one server fails, the other servers can continue to support applications.

3. Differences between relational databases and NoSQL databases

Relational databases have a fixed, predefined schema that all tables must follow, whereas NoSQL databases usually have a dynamic schema that can change easily.

Relational databases contain tables while NoSQL databases like MongoDB contain collections. The data type of each field in the table is fixed for relational databases but it is flexible for NoSQL databases like MongoDB.

Relational databases represent data in tables and rows while NoSQL databases usually store data as collections of documents.

Relational databases commonly use joins to retrieve data across tables. In NoSQL databases like MongoDB, data is often embedded within documents instead of being joined, so joins are used less frequently. As a result, relational databases are generally better suited for complex queries involving multiple related datasets.

4. Applications of relational databases and NoSQL databases

The choice of using a relational or NoSQL database depends on the type of data being stored as well as the nature of tasks that the database is required to perform. In general, relational databases prioritise consistency and structured querying, while NoSQL databases prioritise scalability and performance.

Choose a relational database if:
- The data being stored has a fixed schema.
- Complex and varied queries will be frequently performed.
- The atomicity, consistency, isolation and durability (ACID) properties are critical.
- There will be a high number of simultaneous transactions that require strict data integrity and consistency (ACID compliance), such as in financial or inventory systems.

Choose a NoSQL database if:
- The data being stored has a flexible or evolving schema.
- The system must handle a massive volume of simultaneous read/write operations.
- Data needs to be retrieved or stored with near-instant response times, often for real-time applications.
- There will be an extremely large amount of data that requires horizontal scaling across multiple servers or regions.

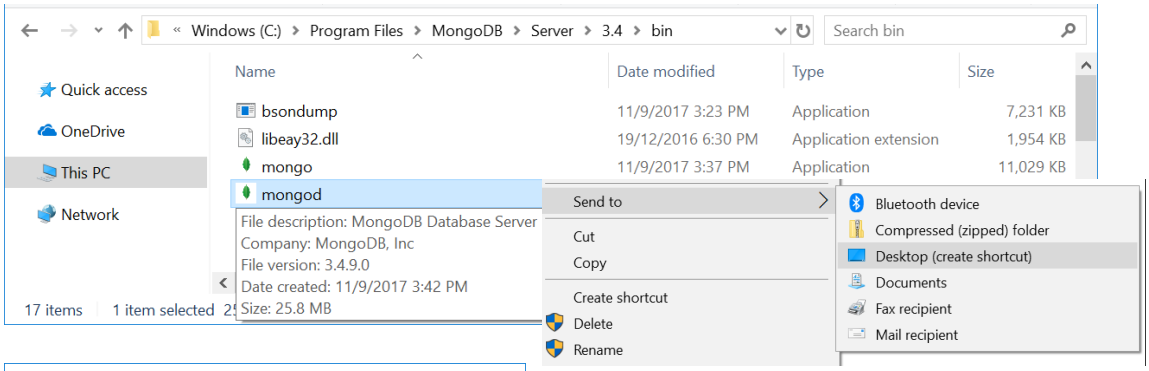
5. Introduction to MongoDB

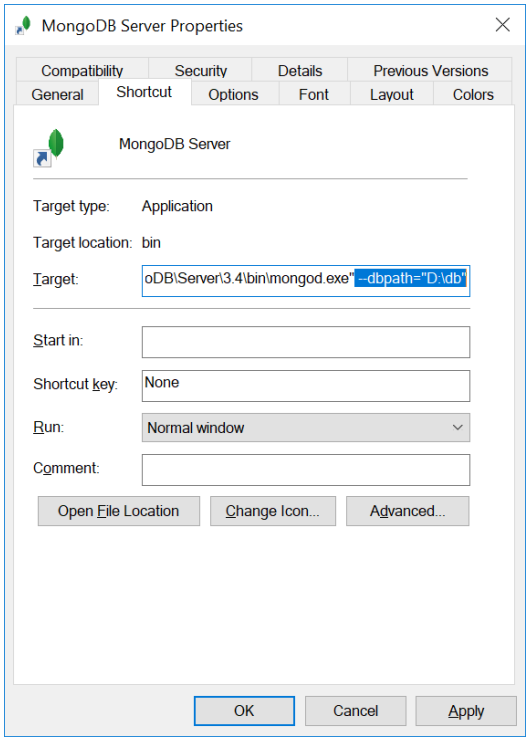
MongoDB is a NoSQL database. Here are the terms in MongoDB with its corresponding terms in relational database.

MongoDB	Relational database
Database	Database
Collection	Table
Document	Row
Field	Column

To start the MongoDB server, click on MongoDB Server icon in the Start menu.

If you don't see the icon, go to the MongoDB installation directory and create a shortcut for **mongod** on the desktop.





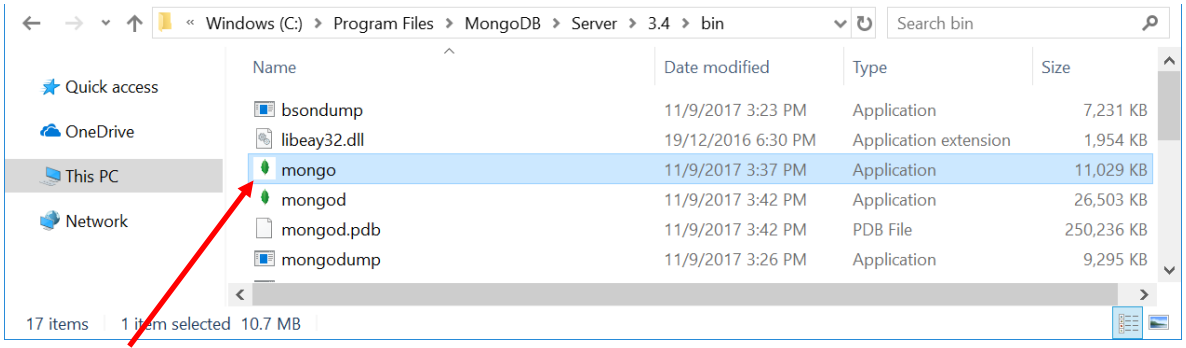
Right-click on the shortcut and select Properties.

Add in `--dbpath="D:\mondb"`, where `D:\mondb` is the directory storing the MongoDB database.

Create this directory to store the database.

Connect to MongoDB server

To interact with MongoDB databases, you need to start a connection to the server.
To do so, use File Explorer and go to the MongoDB directory.



Start **mongo** program.

Starting the mongo shell program allows you to connect to the MongoDB server. You will then enter an interactive shell designed specifically for MongoDB.

```
Select C:\Program Files\MongoDB\Server\3.4\bin\mongo.exe
MongoDB shell version v3.4.9
connecting to: mongodb://127.0.0.1:27017
MongoDB server version: 3.4.9
Server has startup warnings:
2021-05-08T22:48:20.316+0800 I CONTROL  [initandlisten]
2021-05-08T22:48:20.316+0800 I CONTROL  [initandlisten] ** WARNING:
 Access control is not enabled for the database.
2021-05-08T22:48:20.317+0800 I CONTROL  [initandlisten] **
 Read and write access to data and configuration is unrestricted.
2021-05-08T22:48:20.317+0800 I CONTROL  [initandlisten]
>
```

You can do use variables and mathematical operations similar to Python shell.

```
C:\Program Files\MongoDB\Server\3.4\bin\mongo.exe
> 6+7
13
> y=4
4
> 3*4-2
10
> y/3
1.3333333333333333
>
```

Create a MongoDB database

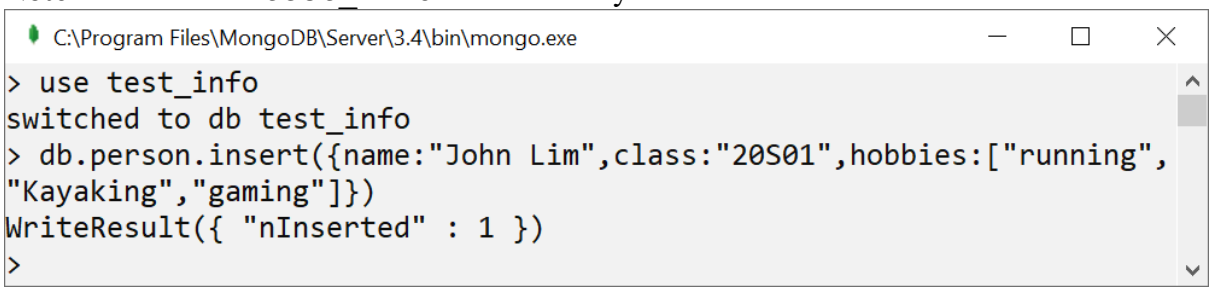
You can create a database with the use statement. You can insert documents into a collection using `insert` statement.

Type the following statements:

```
> use test_info
> db.person.insert({name:"John Lim",class:"20S01",hobbies:["
running","Kayaking","gaming"]})
```

The first statement tells MongoDB to use the database `test_info`. The second statement will insert a document into the `person` collection. Your screen will show that one document has been inserted.

Note: The database `test_info` is created only when a document is inserted.



The collection is automatically created when a document is added. You can manually create the `person` collection using the statement `db.createCollection("person")`.

Type `show dbs` to display the databases that currently have data, and type `show collections` to list the collections inside the database you are using.

1. What happens when you type the following?
   `db.person.insert({name:"Ben",class:"20S01"})`

2. How many documents are there in the `person` collection now?
   Type `db.person.count()` to find out.

3. Type a statement to insert the student `Mary` from class `20S02` with hobby `running` into the `person` collection.

4. Type: db.person.insert({name: "Alex", class: "20S01", age: 16, cca: "Basketball"})? How is this document different from the previous ones in terms of the fields it has?

5. To find all documents, use the `find` function: `db.person.find({})`.

6. To find a particular document, specify the parameters in the `find` function.
   `db.person.find({name:"Ben"})`

7. What documents will this query return?
   `db.person.find({hobbies:"running"})`

8. How do you find all students in class "20S01"?

9. To list all the names of students in class "20S01", use the following:
   `db.person.find({class: "20S01"},{name:1})`

10. How do you get all the names of students in class "20S02"?

11. To get all the names and classes of the students, use the following:
    `db.person.find({},{name:1, class:1})`

12. How do you get all the names and hobbies of the students?

13. Notice that `_id` is always included by default unless excluded:
    `db.person.find({class:"20S01"}, {name:1, _id:0})`

14. To remove the `person` collection, type `db.person.drop()`.

15. To remove the `test_info` database, use the following: `db.dropDatabase()`.

16. To exit Mongo shell, type `quit()`.

17. To shut down MongoDB server, press Ctrl-C.

6. References

Content	Link
NoSQL databases	https://www.thegeekstuff.com/2014/01/sql-vs-nosql-db/ https://www.geeksforgeeks.org/types-of-nosql-databases/ https://www.mongodb.com/resources/basics/databases/types Dan Sullivan (2015). *NoSQL for Mere Mortals*. Person Education.
MongoDB/ PyMongo	https://www.mongodb.com/company/what-is-mongodb https://www.mongodb.com/docs/languages/python/pymongo-driver/current/ https://www.mongodb.com/docs/languages/python/pymongo-driver/current/get-started/